

Efficient Analysis of Overdispersed Data Using an Accurate Computation of the Dirichlet Multinomial Distribution

Sherenaz Al-Haj Baddar¹, Alessandro Languasco², and Mauro Migliardi³

Abstract—Modeling count data using suitable statistical distributions has been instrumental for analyzing the patterns it conveys. However, failing to address critical aspects, like overdispersion, jeopardizes the effectiveness of such an analysis. In this paper, overdispersed count data is modeled using the Dirichlet Multinomial (DM) distribution by maximizing its likelihood using a fixed-point iteration algorithm. This is achieved by estimating the DM distribution parameters while comparing the recent Languasco-Migliardi (LM), and the Yu-Shaw (YS) procedures, which address the well-known computational difficulties of evaluating its log-likelihood. Experiments were conducted using multiple datasets from different domains spanning polls, images, and IoT network traffic. They all showed the superiority of the LM procedure as it succeeded at estimating the DM parameters at the designated level of accuracy in all experiments, while the YS procedure failed to produce sufficiently accurate results (or any results at all) in several experiments. Moreover, the LM procedure achieved a speedup that ranged from 2-fold to 20-fold over YS.

Index Terms—Pattern analysis, overdispersed data, Dirichlet multinomial distribution, log-likelihood function, fixed-point iteration.

I. INTRODUCTION

EFFICIENT analysis of the patterns conveyed by count data has become instrumental in many domains, including, but not limited to, natural language processing, bioinformatics, image classification, and economics. Analyzing count data comprises modeling such data using a suitable statistical distributions. The dispersion (i.e., the scatter-ness) of the count data plays a key role in choosing the distribution that would fit the data the best. For example, the Poisson distribution is suitable for modeling equi-dispersed count data, where the mean and variance are equal. On the other hand, the negative binomial distribution is recommended for overdispersed binary count data in which the variance exceeds the mean. For multicategorical overdispersed count data, the Dirichlet Multinomial

(DM) distribution is recommended. The DM distribution is also popular in Bayesian statistics because it simplifies the mathematical formulation of some commonly-used distributions like the multinomial and categorical distributions, to which it acts as a conjugate prior.

Equation (1) outlines the log-likelihood of the DM distribution, where $\mathbb{N}$ is the set of natural numbers and $\mathbb{N}^* = \mathbb{N} \setminus \{0\}$. Let $K \in \mathbb{N}^*$, $\mathbf{x} := (x_1, \dots, x_K)$, $x_k \in \mathbb{N}^*$ for $k = 1, \dots, K$, $\boldsymbol{\alpha} := (\alpha_1, \dots, \alpha_K)$, $\alpha_k > 0$ for $k = 1, \dots, K$, $N := \sum_{k=1}^K x_k$ and $A := \sum_{k=1}^K \alpha_k$. Let $\psi := 1/A$ be the *overdispersion*¹ parameter and $\mathbf{p} := (p_1, \dots, p_K)$, $p_k > 0$ for $k = 1, \dots, K$, such that $\sum_{k=1}^K p_j = 1$, and set $\alpha_k = p_k/\psi$, $k = 1, \dots, K$. Denoting further as Γ the Euler Gamma function, the *Dirichlet-multinomial log-likelihood* function can be defined as

$$\begin{aligned} \log \mathcal{L}(\mathbf{p}, \psi, \mathbf{x}) := & - \left(\log \Gamma \left(\frac{1}{\psi} + N \right) - \log \Gamma \left(\frac{1}{\psi} \right) \right) \\ & + \sum_{k=1}^K (\log \Gamma(\alpha_k + x_k) - \log \Gamma(\alpha_k)). \end{aligned} \quad (1)$$

Modeling a dataset using a statistical distribution comprises estimating the parameters that maximize its likelihood. The work in [17] offers an iterative procedure that utilizes a closed form for estimating the maximum-likelihood of the DM distribution. This procedure utilizes the DM log-likelihood function which is known for its instability when the data is overdispersed, i.e., when the overdispersion parameter ψ approaches 0. This instability is attributed to the fact that when the data is overdispersed and/or when the counts themselves are too big, the paired differences of the $\log \Gamma$ functions used in (1) become too small; thus, prone to errors due to the limited accuracy of computer-based floating-point arithmetic [13]. The solution proposed in [28] has been utilized in the literature; examples include [5], [8], [25], [27], [30], however, it suffers from serious flaws that jeopardize both its soundness and its efficiency. An efficient solution that provides the desired level of accuracy in computing the log-likelihood estimates of the DM distribution is outlined in [13].

This study aims at modeling overdispersed categorical count datasets by estimating the parameters of the DM distribution.

¹We take this occasion to correct a typo occurred in [13]: on page 2, this word is written as “overdisposition” which is clearly wrong.

Received 2 September 2023; revised 13 February 2024; accepted 26 October 2024. Date of publication 31 October 2024; date of current version 9 January 2025. Recommended for acceptance by M. Mahoney. (Corresponding author: Alessandro Languasco.)

Sherenaz Al-Haj Baddar is with the Department of Computer Science, The University of Jordan, Amman 11942, Jordan (e-mail: s.baddar@ju.edu.jo).

Alessandro Languasco is with the Dipartimento di Ingegneria dell’Informazione, Università di Padova, 35131 Padova, Italy (e-mail: alessandro.languasco@unipd.it).

Mauro Migliardi is with the Dipartimento di Ingegneria dell’Informazione, Università di Padova, 35131 Padova, Italy (e-mail: mauro.migliardi@unipd.it). Digital Object Identifier 10.1109/TPAMI.2024.3489645

It employs the iterative fixed-point algorithm described in [17] while comparing the results achievable using two different procedures for estimating the **DM** log-likelihood function; the first procedure is depicted in [13], and is denoted by **LM**, while the second procedure is outlined in [28] and is denoted by **YS**. The results that can be obtained are compared in terms of their speed, accuracy, and the goodness-of-fit using multiple overdispersed datasets.

II. THE ESTIMATION OF THE DIRICHLET MULTINOMIAL DISTRIBUTION PARAMETERS

The main goal of this study is to present an efficient and accurate approach to estimating the Dirichlet multinomial distribution parameters in order to model overdispersed categorical count data. In order to estimate the Dirichlet distribution parameter α , a subset of the input is selected at random. Next, an iterative algorithm is applied to this subset in order to estimate α to a preset degree of accuracy [17]. In this work, the fixed-point algorithm described in [17] and illustrated in Algorithm 1 is employed, where F denotes the *digamma* function.²

In addition to a randomly chosen subset of the original dataset with R rows and K columns, denoted by **data**, the fixed-point algorithm requires an initial guess for the α parameter. It also utilizes a maximum number of iterations, denoted by $\mathcal{T}$, and an upper limit (i.e., a threshold) on the acceptable difference between two consecutive estimates of α , denoted by Δ . As depicted in Algorithm 1, no more than $\mathcal{T}$ iterations are executed, in each iteration a new estimate for the parameter α , denoted by α_1 , is generated. This is achieved by scanning **data** in order to calculate the quantities Σ_1 and Σ_2 which accumulate two paired differences of the digamma function F , as described in (55) in [17]. These two quantities are then used to estimate a new α (i.e., α_1). Next, the absolute paired difference of the log-likelihood of the **DM** distribution at both α_0 and α_1 is calculated and compared to the threshold Δ ; if that difference is less than Δ , then the algorithm is assumed to have converged. In this case, it returns α_1 and the accompanying parameter ψ_1 . However, if no such convergence is reached by $\mathcal{T}$ iterations, the algorithm is assumed to have failed and the value α_{error} is returned.

The soundness of the fixed-point iteration is jeopardized when the data is overdispersed since the paired difference, depicted in line 14, comprises not one, but two calls to the **DM** log $\mathcal{L}$ function. Each such call is prone to the lack-of-accuracy problem detailed in [13]. Two possible solutions are described in detail in [13] and [28]. Sections II-A and II-B shed light on them.

A. The Yu-Shaw Procedure

Following the notation used in [28], we recall that Yu and Shaw reduced the problem of computing $\log \mathcal{L}(\mathbf{p}, \psi, \mathbf{x})$ to the one pertaining the evaluation of $\log \Gamma(1/x + y) - \log \Gamma(1/x)$,

²In the literature the digamma function $F(u)$ is also denoted as $\psi(u)$ but in this context ψ has a different meaning.

Algorithm 1: A Fixed-Point Iteration Algorithm for Estimating the **DM** Parameter α .

ESTIMATE- α (**data**, $\mathbf{x}$, $\mathcal{T}$, R , K , Δ , α_0)

- **data**: a subset of the original dataset; a matrix of counts with R rows and K columns
- $\mathbf{x}$: a vector of length K , where x_k is the summation of all counts in column k in **data**
- $\mathcal{T}$: maximum number of iterations until convergence
- Δ : an upper limit on the difference between two consecutive estimates of α
- α_0 : initial guess for α
- returns (α, ψ) , the **DM** distribution parameters if ESTIMATE- α converges, otherwise, returns α_{error}

```

1:  for  $j = 1, \dots, \mathcal{T}$ 
2:     $\Sigma_1 \leftarrow 0; \Sigma_2 \leftarrow 0$ 
3:     $\Sigma_{\alpha_0} \leftarrow \sum_{k=1}^K \alpha_0[k]$ 
4:    for  $k = 1, \dots, K$ 
5:      for  $i = 1, \dots, R$ 
6:         $\Sigma_i \leftarrow \sum_{k=1}^K \mathbf{data}[i, k]$ 
7:         $\Sigma_1 \leftarrow \Sigma_1 + F(\mathbf{data}[i, k] + \alpha_0[k]) - F(\alpha_0[k])$ 
8:         $\Sigma_2 \leftarrow \Sigma_2 + F(\Sigma_i + \Sigma_{\alpha_0}) - F(\Sigma_{\alpha_0})$ 
9:      endfor  $i$ 
10:    $\alpha_1 \leftarrow \alpha_0 \cdot (\Sigma_1 / \Sigma_2)$ 
11:   endfor  $k$ 
12:    $\psi_0 \leftarrow 1 / \sum_{k=1}^K \alpha_0[k]; \mathbf{p}_0 \leftarrow \psi_0 \cdot \alpha_0$ 
13:    $\psi_1 \leftarrow 1 / \sum_{k=1}^K \alpha_1[k]; \mathbf{p}_1 \leftarrow \psi_1 \cdot \alpha_1$ 
14:   if  $(|\log \mathcal{L}(\mathbf{p}_0, \psi_0, \mathbf{x}) - \log \mathcal{L}(\mathbf{p}_1, \psi_1, \mathbf{x})| < \Delta)$ 
15:     return  $[\alpha_1, \psi_1]$ 
16:   else
17:      $\alpha_0 \leftarrow \alpha_1$ 
18:   endfor  $j$ 
19:   return  $\alpha_{\text{error}}$ 

```

where $x > 0$ and $y \in \mathbb{N}^*$, using

$$\begin{aligned} \log \Gamma\left(\frac{1}{x} + y\right) - \log \Gamma\left(\frac{1}{x}\right) &= -y \log x + D_{\infty}(x, y) \\ &= -y \log x + D_m(x, y) + e_m(x, y), \end{aligned} \quad (2)$$

where $x(y-1) < 1$, $m \in \mathbb{N}$, $m \geq 2$,

$$\begin{aligned} e_m(x, y) &:= \sum_{\ell=m+1}^{\infty} \frac{(-1)^{\ell} \phi_{\ell}(y)}{\ell(\ell-1)} x^{\ell-1}, \\ D_m(x, y) &:= \sum_{\ell=2}^m \frac{(-1)^{\ell} \phi_{\ell}(y)}{\ell(\ell-1)} x^{\ell-1} \end{aligned} \quad (3)$$

and $D_{\infty}(x, y) := \lim_{m \rightarrow \infty} D_m(x, y)$. Moreover, thanks to Faulhaber's formula, we have

$$\phi_n(u) := n \sum_{k=0}^{u-1} k^{n-1} = B_n(u) - B_n(0), \quad (4)$$

$u \in \mathbb{N}^*$, where $B_n(u)$ are the *Bernoulli polynomials* of order $n \geq 2$. The polynomials $\phi_n(u)$ are also called the *old-Bernoulli*

polynomials of order $n \geq 2$. We remark that the condition $x(y - 1) < 1$ is needed to ensure the absolute convergence $D_\infty(x, y)$ in (2) and that in [28, eq. (13)] Yu and Shaw sketched a faulty proof of (2). This formula was in fact proved in [13, Section II] using the Taylor-MacLaurin formula for $\log(1 + u)$ and a straightforward argument.

To handle the computation of (2) with a sufficiently good accuracy, in [28] the value $\delta = 1/5$ is chosen and the $[1/x, 1/x + y]$ interval is split in subintervals $[a, a + b]$ such that $a > 0$, $b \in \mathbb{N}^*$ and $b/a \leq \delta$. In this way, the computation is performed on a *mesh* of points that corresponds to the collection of the starting and ending points of such subintervals. Moreover, Yu and Shaw choose $m = 20$ and hence claimed that the size of $|e_{20}(1/a, b)|$, $b/a \leq \delta = 1/5$, is “good enough” for their goals.³ But they did not consider the error-accumulation phenomenon that comes from repeating the computation in every subinterval of the previously described type.

To be more precise, the goal of the mesh generating procedure is to reach $1/x + y$ starting from $1/x$ by building a sequence of subintervals $[z_{j-1}, z_j] \subseteq [1/x, 1/x + y]$, $z_{j-1} < z_j$, $\bigcup_{j=1}^L [z_{j-1}, z_j] = [1/x, 1/x + y]$, $L \in \mathbb{N}^*$, in which we can use formula (2) with a “sufficiently good accuracy” (which depends on δ), i.e., such that $y_j := z_j - z_{j-1} \in \mathbb{N}^*$ and $y_j/z_{j-1} \leq \delta$ for every $j = 1, \dots, L$. The set of such z -points are the mesh of points used by Yu-Shaw.

The first step in the mesh generating procedure is then to ensure that we can use (2) with a “good accuracy” in the first subinterval, thus in $[1/x, 1/x + y_1]$, for some $y_1 \in \mathbb{N}^*$. So one needs to choose the largest natural number $y_1 \geq 1$ such that $xy_1 \leq \delta$. The solution is clearly⁴ $y_1 = \lfloor \delta/x \rfloor$ and the fact that $y_1 \geq 1$ is equivalent to $\delta/x \geq 1$ (so that $x \leq \delta$). If $x > \delta$ we have $y_1 = 0$ and the whole mesh generating procedure is aborted; in other words, if $x > \delta$, we will for sure fail our goal of reaching $1/x + y$ starting from $1/x$. Hence, the condition $z_0 = 1/x \geq 1/\delta$ is mandatory to ensure that z_j is a strictly increasing sequence and that $y_j \in \mathbb{N}^*$ for every $j = 1, \dots, L - 1$.

We then have to insert (2) into (1); so one has to let x play the role of ψ and $1/\alpha_k$, $k = 1, \dots, K$. From the crucial condition $x \leq \delta$ we saw before, one immediately gets that the *Yu-Shaw mesh generating procedure works if and only if* $\alpha_k \geq 1/\delta$ for every $k = 1, \dots, K$. In this case, recalling that $\psi = 1/\sum_{k=1}^K \alpha_k$, one also obtains that $\psi \leq \delta/K \leq \delta$.

We will refer to this phenomenon as the δ -barrier. Hence the *Yu-Shaw procedure in [28] works only for datasets that do not break the δ -barrier*.

To avoid this problem one has to replace (2) with a different way of computing paired differences of $\log \Gamma$ -values; this is what Languasco-Migliardi did in their paper [13].

B. The Languasco-Migliardi Procedure

In 2023 Languasco and Migliardi [13] greatly improved on the **YS**-approach by replacing (2) with another result proved

using the Euler-MacLaurin summation formula, see [13, Theorem 1]. In this way they were not only able to remove the δ -barrier problem from the picture, but they also improved the computational cost of the whole $\log \mathcal{L}$ -evaluation procedure, see [13, Section 4.2]. In fact, they reduced both the number of steps needed to obtain the final result with the user-desired accuracy and the number of products needed for achieving the same goal.⁵ Moreover, they were able to carefully estimate the *error-accumulation* phenomenon due to the fact that the computation of paired $\log \Gamma$ -values in (1) has to be repeated several times, [13, Section 4.1]; a fact never mentioned into [28].

They also showed how, see [13, Section 5.2], to decide if the final accuracy of `prec` decimal digits in the mantissa (required by the user) can be obtained by performing the whole computation using a predetermined floating-point number type. In this way, they also gave a *reliability*-criterion of the overall procedure; a feature never showed in any previous approach.

Clearly, we cannot here repeat the whole algorithm presented in [13], so we just further specify that a way of guessing the order of magnitude of $\log \mathcal{L}$ for an overdispersed dataset is to evaluate the *asymptotic state of the system* defined as follows.

Definition (Asymptotic state of the system) We define the *asymptotic state of the system* as $\psi \rightarrow 0^+$ as

$$\mathcal{A} := \lim_{\psi \rightarrow 0^+} \log \mathcal{L}(\mathbf{p}, \psi, \mathbf{x}) = \sum_{k=1}^K x_k \log p_k.$$

This definition is coherent with [28, eq. (20)], and with [13, Corollary 1]. It is clear that $\mathcal{A}$ only depends on the dataset probabilities $\mathbf{p}$ and data counts $\mathbf{x}$; so $\mathcal{A}$ depends only on some dataset characteristics.

In practice, if a user requires that the value of $\log \mathcal{L}(\mathbf{p}, \psi, \mathbf{x})$ should be determined with an accuracy of `prec` mantissa digits using the floating-point `double`-type of the C-programming language, we first evaluate $\mathcal{A}$ and then we check if `prec` + 2 + $\lceil \log_{10} |\mathcal{A}| \rceil \leq 15$, since the largest representable integer of the `double`-type has 15 decimal digits. If not, we ask the user to lower the value of `prec`, or to use an implementation that allows the use of larger numbers. In this procedure, a possible alternative is to replace $\mathcal{A}$ with $\mathcal{B} := \sum_{k=1}^K x_k \log f_k$, where $f_k := x_k/N$, $k = 1, \dots, K$, are the frequencies of each category. In this way, both the quantities $\mathcal{A}$ and $\mathcal{B}$ are useful in predetermining if the floating-point arithmetic will be able to handle the computation of $\log \mathcal{L}(\mathbf{p}, \psi, \mathbf{x})$ with the user-required accuracy. Remark that this result-reliability problem does not depend on the approach we are using to compute $\log \mathcal{L}$ but it is an intrinsic property of the dataset we are working with, and is also relevant to the data-type used in the actual implementation.

From now on, we will refer to the way of computing $\log \mathcal{L}$ that uses Languasco-Migliardi [13] results coupled with the result-reliability idea previously showed, as the **LM**-procedure. For more details on the formulae involved in it, we suggest the reader to consult [13].

³But their estimate of $|e_m(x, y)|$ in [28, eq. (15)] is wrong, see [13, Section II]. Its correct estimate is proved in [13, eq. (21)].

⁴By $\lfloor u \rfloor$ we denote the integral part of the real number u .

⁵They essentially passed from a number of products that quadratically depends on the number of steps (**YS**-approach) to a linear function in the number of steps (**LM**-approach).

TABLE I
DATASETS

Dataset	Type	Categories	Instances	ψ	Δ
Trump-Impeachment-polls (TIP) ⁶	Count	3	408	0.023761	$2.50 \cdot 10^{-8}$
Trump-Clinton-polls (TCP) ²	Count	2	300	0.005848	$3.33 \cdot 10^{-8}$
Cane ³	Image	120	85	0.000432	0.000826
Cavallo ⁴	Image	120	130	0.000797	0.000826
IoT-Camera (I) (IoT-C-I) ⁵	Count	5	3420	0.068320	$1.67 \cdot 10^{-8}$
IoT-Camera (II) (IoT-C-II)	Count	5	200	0.007249	$1.67 \cdot 10^{-8}$

¹<https://www.kaggle.com/datasets/teejmahal20/trump-impeachment-polls>²<https://www.kaggle.com/datasets/etsec9287/2020-general-election-polls>³<https://www.kaggle.com/datasets/alessiocorrado99/animals10>⁴<https://www.kaggle.com/datasets/alessiocorrado99/animals10>⁵<https://data.mendeley.com/datasets/84cc8grtk/1>

III. EXPERIMENTATION AND RESULTS

This section depicts the results of a set of experiments conducted in order to provide a clear comparison of the results that can be achieved with two different implementations of Algorithm 1: one using **LM** procedure and one using **YS** procedure. The experiments, mostly using overdispersed data, compare their performance both in terms of the number of iterations and in terms of the time to converge. Moreover, the closeness of the model, defined by the estimated parameters α and ψ , to the real model is evaluated using a set of distance metrics as well as two statistical goodness-of-fit tests. Since count data is assumed to be overdispersed if the value of ψ is close to 0, in the context of this study, a dataset is assumed to be overdispersed if $\psi < 0.05$.

The experiments showed that the **LM** procedure outperformed **YS** in each and every case. Moreover, **YS** failed to converge in multiple experiments and/or suffered from the δ -barrier problem, as defined in Section II-A. As a consequence, the **LM** procedure was solely utilized for tuning the parameters associated with some datasets, like the number of categories and tolerance. Moreover, the **YS** log-likelihood results were not used in the models' closeness experiments due to their potential lack of accuracy.

First the datasets employed are highlighted, then the experiments that compare both procedures are discussed. Finally, the models' closeness in terms of the parameters estimated by **LM** is evaluated and discussed.

A. Datasets

In this study, five public categorical datasets were utilized. Aside from overdispersion, these datasets, as illustrated in Table I, vary in size, type, and number of categories. Each of them has its pre-defined tolerance value, Δ , that helps determine if the fixed-point iteration has converged. The Trump-Impeachment-polls and Trump-Clinton-polls datasets comprise vote counts, while Cane and Cavallo are both image datasets. The IoT-Camera (I) dataset was obtained by processing the D-Link IoT devices dataset which comprised packet traces collected from a set of security cameras. One of the cameras' packet traces were aggregated over a fixed time interval according to the packets protocol category, resulting in a set of 3420 count vectors. Due to the fact that the **YS** procedure failed to converge in every test of this dataset, 200 instances were selected randomly from the

TABLE II
NUMBER OF ITERATIONS TO CONVERGE, (IC), FOR **LM** AND **YS**

Dataset	5%		10%		15%		20%	
	LM	YS	LM	YS	LM	YS	LM	YS
TIP	1746	1746	1725	1725	1831	1831	1882	1883
TCP	6139	7012	7262	7263	6620	6621	8394	8397
Cane	1682	1556	1044	1032	946	895	1071	1048
Cavallo	688	662	530	513	528	505	413	394
IoT-C-I	508	NA	475	NA	471	NA	488	NA
IoT-C-II	1943	1934	1698	1698	2243	2243	2711	2711

original dataset, and were considered a separate dataset, referred to as IoT-Camera (II).

According to the assumption stated earlier in this section, all datasets in Table I, with the exception of IoT-C-I, are overdispersed. To convert the Cane and Cavallo datasets to count data, a bag-of-visual-words approach was employed to transform each image into multiple count vectors. A Python script⁶ that implements this approach was applied to the original images after specifying the desired number of categories, which was set at 120. This number was chosen as it manifested the overdispersed nature of the images, while keeping the time to converge reasonable for experimentation. As illustrated in Algorithm 1, the fixed-point iteration continued until either the maximum number of iterations was reached, or until the difference between two consecutive estimates for α was less than the tolerance value Δ . Equation (5) illustrates the formula used for calculating Δ :

$$\Delta := \frac{10^{-\text{prec}-2}}{K+1}, \quad (5)$$

where prec denotes the level of accuracy of the log-likelihood value using the **LM** procedure, in terms of the decimal digits of the mantissa, and K designates the number of categories. It is worth noticing that the tolerance values for the Cane and Cavallo datasets were each increased by a factor of 10^6 to keep the time to converge reasonable for experimentation.

B. Experimentation Setup

A script that implements the fixed-point iteration algorithm was developed using both **LM** and **YS** in Python (3.10.2) with numpy (1.22.2), mpmath (1.2.1), and scipy (1.8.0) packages. The paired differences of the F function in Algorithm 1, lines 7 and 8, were computed using the C-compiled `digamma(x)` function implementation provided in the scipy package. The experiments were executed on an Intel Core i5 computer with 8GB of RAM. The euclidean distance metric, denoted by **D**, was calculated using the `LA.norm` function from the numpy package, whereas the χ^2 -test was calculated using the `scipy.stats.chisquare` function.

C. Iterations to Converge

Table II depicts the results of a set of experiments conducted in order to contrast **YS** and **LM** in terms of the number of iterations

⁶<https://colab.research.google.com/github/pinecone-io/examples/blob/master/learn/image-retrieval/bag-of-visual-words/bag-of-visual-words.ipynb>

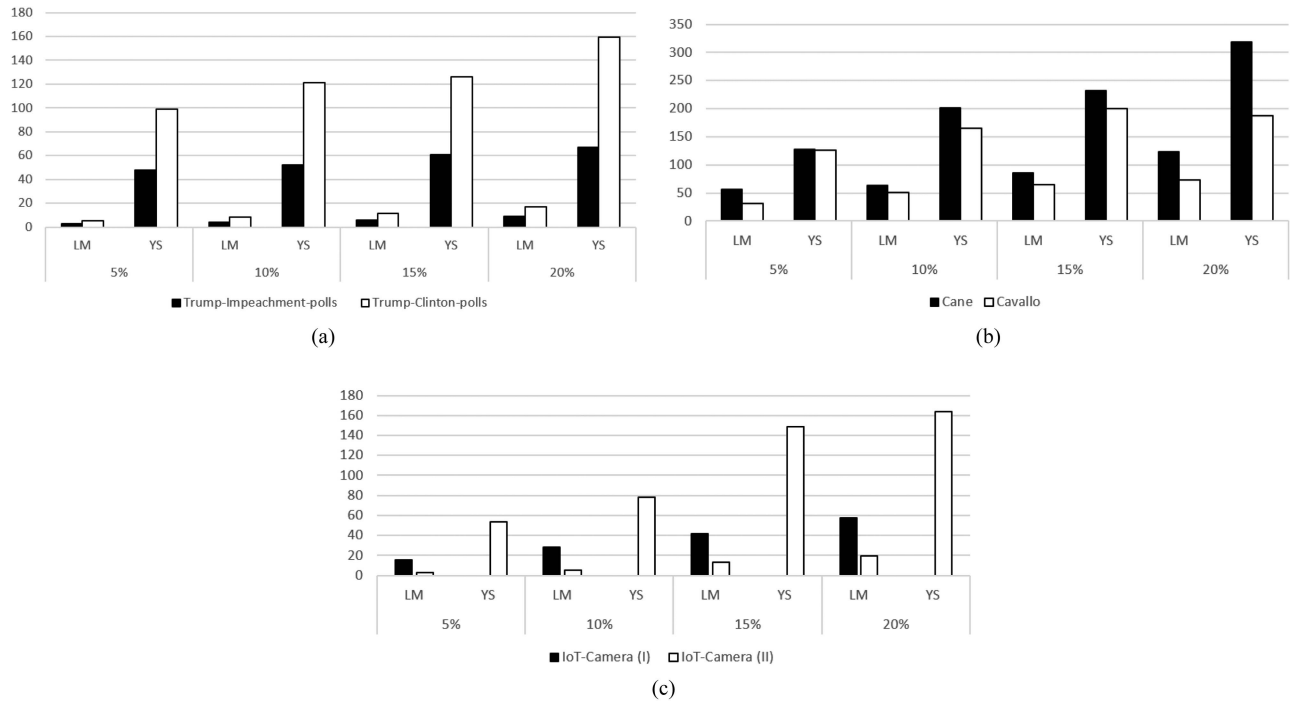


Fig. 1. Time to Converge (TC) for both **LM** and **YS** (seconds). (a) Time to Converge (TC) for both **LM** and **YS** (seconds). (b) Time to Converge (TC) for both **LM** and **YS** (seconds). (c) Time to Converge (TC) for both **LM** and **YS** (seconds).

to converge (denoted by IC). To measure the impact of the input size, the experiments were repeated using randomly selected subsets that comprised 5%, 10%, 15%, and 20% of each of the original datasets; each experiment was repeated five times, each with a different subset of the original data, and the results were then averaged.

It is worth noting that for the Cane and Cavallo datasets, the **YS** algorithm failed to produce accurate log-likelihood estimates due to failing to meet the δ -barrier condition mentioned in Section II-A. Thus, this condition had to be suppressed in order to get **YS** to converge. However, this deemed its respective log-likelihood estimates inappropriate for models' closeness evaluations due to their lack of accuracy. The experiments in which this situation was encountered have their respective results underlined in Table II. For the Trump-Impeachment-polls dataset with 1224 data points, the IC value required by **LM** and **YS** ranged from 1746 for 5% of the dataset, up to 1883 iterations for 20% of the dataset. For the IoT Cameras (II) dataset, these numbers ranged from 1934 to 2711, respectively. For the Trump-Clinton-polls dataset, it ranged from 6139 for **LM** and 7012 for **YS** using 5% of the dataset, up to about 8397 iterations for 20% of the dataset. It can be observed that for a given dataset, IC increases as the input size increases. However, the results depicted in Table II imply that neither ψ nor the number of categories, K , seem to be directly correlated to IC . For example, the Trump-Clinton-polls dataset consumed a significantly larger number of iterations to converge compared to the four other datasets, although it only has 2 categories, and comprises only 600 data points. For 20% of the dataset, the IC value for the Trump-Clinton-polls dataset was more than 20 times larger than the corresponding quantity for the Cavallo dataset which features

the most data points, and the most categories. To further support this observation, the IC values for both the Cane and Cavallo datasets can be examined; for the Cane dataset which features almost 10K data points and an overdispersion index ψ of almost 0.000432, the IC values to converge at 20% of the dataset were 1071 and 1048 for **LM** and **YS**, respectively. On the other hand, for the Cavallo dataset with almost 15.5K data points and a ψ -value of 0.000797, the corresponding values were 413 and 394 for **LM** and **YS**, respectively, also at 20% of the dataset.

D. Time to Converge

The number of iterations to converge does not necessarily reflect the amount of time to converge (TC) for the fixed-point algorithm. Fig. 1(a) through (c) illustrate the time it takes **LM** and **YS** to converge for each of cases depicted in Table II. It shows that the **LM** procedure significantly outperforms **YS** in terms of the time to converge. Fig. 1 also shows that, in general, increasing the number of data points implies increasing TC . In Fig. 1(b), it can be noticed that it took both **LM** and **YS** more time to converge for the datasets with 10K and 15.5K data points compared to smaller ones. The gap between **LM** and **YS** in terms of TC is evident in all cases; as Fig. 1(a) and (c) show, at 5% of the original dataset, **LM** was 20 times faster to converge than **YS** for the smaller datasets, while at 20% of the dataset, **LM** was about 8.5 times faster to converge than **YS** for the same datasets. Fig. 1(b) shows that for the Cane dataset, **LM** was almost 2 times faster than **YS** to converge at 5% of the dataset, and almost 2.5 times faster at 20% of the dataset. For the Cavallo dataset, **LM** was 4 times faster than **YS** to converge at 5% of the dataset, and almost 2.5 times faster at 20% of the dataset.

TABLE III
DISTANCE AND GOODNESS-OF-FIT FUNCTIONS

Function	Description
Total Variation Distance (tvD)	$\text{tvD}(P, Q) := \frac{1}{2} \sum_{k=1}^K P[k] - Q[k] $
KL-Distance (KL-D)	$\text{KL-D}(P, Q) := \sum_{k=1}^K (P[k] \cdot \log(\frac{P[k]}{Q[k]}))$
Euclidean Distance (D)	$\text{D}(P, Q) := \sqrt{\sum_{k=1}^K (P[k] - Q[k])^2}$
Mean-Square-Error (MSE)	$\text{MSE} := \frac{1}{K} \sum_{k=1}^K (P[k] - Q[k])^2$
Chi-squared Test (χ^2 -test)	$\chi^2\text{-test}(O, E) := \sum_{k=1}^K \frac{(O_k - E_k)^2}{E_k}$
The Z-test (Z-test)	$Z\text{-test}(O, E) := \frac{\bar{x} - \mu}{\sigma}$

Fig. 1(b) also shows that the Cane dataset which features fewer data points (i.e., 10K), but a closer to zero overdispersion index of $\psi = 0.000432$, required both the **LM** and **YS** procedures more time to converge compared to the Cavallo dataset which featured 15.5K data points, and an overdispersion index larger by a factor of 1.8. Using **LM** at 5% and 20% of the original dataset, the *TC* value for the smaller Cane dataset exceed that for the larger Cavallo dataset by a factor of almost 1.7. It is worth-noting that while the speedup **LM** achieved over **YS** for these two datasets is smaller, relatively speaking, it is still quite significant given the fact that the **YS** algorithm results are unreliable. In Fig. 1(c), no results were reported for **YS** using the IoT-Camera (I) dataset as it failed to converge at 300 seconds or less. Additionally, for the larger IoT-Camera (I) dataset, it took Algorithm 1 much less *IC* value compared to the IoT-Camera (II) dataset. Yet, it required significantly longer *TC*. This implies that the number of iterations to converge by itself is not sufficient to predict the relative speed at which such an algorithm would converge.

E. Goodness of Estimates

The time it takes Algorithm 1 to converge is not enough to assess the quality of the estimated **DM** distribution parameters; the closeness of the model implied by these parameters to the real one is crucial for the significance of the whole process. Therefore, a set of distance metrics was employed to evaluate the goodness of the estimated α and ψ values, that includes: Total Variation Distance (**tvD**), the Kullback-Leibler divergence Distance (**KL-D**), euclidean Distance (**D**), and the Mean-Square-Error (**MSE**). Additionally, two statistical goodness-of-fit tests were applied to assess whether the estimated parameters produce a model close enough to the original model or not. Table III illustrates the distance and goodness-of-fit functions utilized in this study.

Where:

- P and Q denote the real and expected probability vectors, respectively;
- O and E denote the real (i.e., observed) and expected counts vectors, respectively;
- $\bar{x}$ denotes the mean of the expected counts vector, while μ denotes the mean of the real counts vector, and σ denotes its standard deviation.

TABLE IV
ERROR ESTIMATES IN ψ

Dataset	ψ	tvD	KL-D	D	MSE
TIP	0.023761	0.016644	0.000769	0.020815	0.000160
TCP	0.005848	0.004330	0.000044	0.006123	0.000021
Cane	0.000432	0.051405	0.008288	0.012543	0.000001
Cavallo	0.000797	0.035087	0.004199	0.008713	0.000001
IoT-C-I	0.068320	0.046693	0.013705	0.053475	0.000575
IoT-C-II	0.007249	0.009707	0.000357	0.010468	0.000024

TABLE V
THE STATISTICAL TESTS χ^2 -TEST AND Z-TEST RESULTS

Dataset	ψ	χ^2 -test	Z-test
TIP	0.023761	pass	pass
TCP	0.005848	pass	pass
Cane	0.000432	NA	pass
Cavallo	0.000797	NA	pass
IoT-C-I	0.068320	fail	pass
IoT-C-II	0.007249	pass	pass

While **tvD**, **KL-D**, **D**, and **MSE** are utilized to calculate an estimate of the error in the **DM** distribution parameter ψ , χ^2 -test and Z-test are utilized to test whether the expected probabilities implied by ψ are close enough to the actual probabilities implied by the dataset as a whole. The distance functions in Table III were computed for each of the experiments depicted in Fig. 1, resulting in the estimates illustrated in Table IV. The results of applying the χ^2 -test and Z-test are presented in Table V.

The lowest **tvD** and **KL-D** values, at 0.004330, and 0.000044, respectively, were reported for the parameter estimates of the Trump-Clinton-polls dataset. That dataset also achieved the lowest **D** value of 0.006123. The highest **tvD** error value was reported for the parameter estimates of the Cane dataset, which scored 0.051405, while the IoT-Camera-I dataset scored the highest **KL-D** and **D** errors at 0.013705 and 0.053475, respectively. The **MSE** in parameter estimates reported across all datasets were, in general, equal to 10^{-5} or less. This implies its limited effectiveness in error estimation, in the context of modeling overdispersed count data. To further evaluate the goodness of the estimated values of ψ , the χ^2 -test and Z-test were employed. The χ^2 -test measures how close a vector of expected counts is to the real (i.e., observed) counts vector, while the Z-test is used to decide whether the means of two datasets are different when the variances are known. The χ^2 -test fails when any count in either vector, expected or real, is below 5. Thus, the Z-test was also utilized. If at least one of the two tests is passed, then the two count vectors are considered to belong to the same distribution, and this implies that the **DM** estimated parameters succeeded at modeling the original data. As a consequence, the approach depicted in this study does not suffer from overfitting, a situation encountered when the model proposed to capture the distribution of a given population using a sample of that population fails to recognize unseen population data points that were not used in generating the model. As Table V shows, both tests were successful in all examined cases, except for the ones in which

the counts were less than 5. In these cases, the χ^2 -test was not applicable, unlike the Z -test which was successfully applied. This situation was encountered in the Cane and Cavallo datasets, and hence, their χ^2 -test entries in Table V are labeled as **NA** for Not Applicable. For the IoT-Camera (I) dataset, the χ^2 -test, unlike the Z -test, failed as it did not manage to consider the real and expected count vectors to be close enough.

IV. RELATED WORK

The process of estimating the parameters of the Dirichlet Multinomial (**DM**) distribution has been known to be inefficient. Moreover, overdispersed count data complicate this process further by jeopardizing its soundness. Thus, several studies attempted to offer efficient and accurate mechanisms for estimating the **DM** distribution parameters by maximizing its log-likelihood function. Different approaches for maximum log-likelihood estimation have been cited in the literature including: scoring functions, Expected Maximization (**EM**), and neural networks. However, recent studies opted for the Newton-Raphson method in order to achieve this goal.

The study depicted in [23] utilized the Newton-Raphson method for estimating the **DM** distribution parameters, and shed light on several limitations in this method. However, it lacked rigorous evaluation of the errors in the estimates. Besides, the study did not offer comparisons with other similar literature, and failed to address the criticality of overdispersed count data. The Newton-Raphson method has been utilized for the **DM** parameter estimation in other recent studies, as well [15], [20], [24], [29].

The study in [19] employed a scaled Dirichlet distribution in the context of software modules categorization. To estimate the **DM** distribution parameters, the Expected Maximization approach was utilized. This approach was also utilized for the **DM** parameter estimations in other studied [6], [9], [32].

The work in [1] utilized a gradient ascent approach to estimating a set of parameters for a mixture of Dirichlet distributions employed for topic detection in short text. This technique is also utilized, among others, to estimate the parameters of the **DM** distributions and its variants in the context of document classification in [2]. Other similar work is depicted in [14], [16], and [7].

The work in [3] proposed a Variational Autoencoder for modeling topics that utilized the **DM** distribution as a prior in order to account for sparsity. Their approach comprised a mechanism for estimating the Dirichlet distribution overdispersion parameter using a neural network: the input data was first suitably transformed using such a neural network and then a linear transformation was applied to produce an intermediate quantity that helped estimate the **DM** parameter. Another example is depicted in [4], where recurrent neural networks were utilized in order to estimate the parameters of Latent Dirichlet Allocation (LDA) and Hierarchical Dirichlet Processes (HDP) utilized in speech-based recognition of genre and topics for a set of broadcast shows. Other recent studies on utilizing neural networks for estimating the **DM** distribution parameters can be found in [12], [21] and [22].

In some works the log-likelihood function of the **DM** distribution is approximated, see, e.g., [6], [10], [31], by letting some

of the main variables tend to zero or to infinity. In practice, though, these parameters are fixed and positive and depend on the used dataset. Moreover, the introduced approximation errors are estimated only on a statistical basis and hence it is not possible to compare our results with theirs. In other cases, see, e.g., [11], a zero-inflation procedure is performed over the data, but it is unclear how this would change the behaviour of the **DM** log-likelihood function on the used datasets.

The present study differs from previous works in that it chose the fixed-point iteration method to help estimate the **DM** distribution parameters. Moreover, it introduced a predetermined reliability-criterion for the whole procedure, see Section II-B. This choice was also motivated by the fact that the **EM** and Newton-Raphson approaches suffer from limitations in the context of solving this problem. The **EM** approach is known to be computationally demanding, and prone to convergence to local optima. Moreover, the quality of the estimates it calculates depends on the parameter's initial guess. The Newton-Raphson approach is not always applicable, can be computationally demanding, and is also prone to convergence to local optima. Moreover, the initial guess of its parameter has to be close to its real value.

V. CONCLUSIONS AND FUTURE WORK

In this study, an efficient and accurate technique for estimating the Dirichlet Multinomial (**DM**) distribution parameters for overdispersed count data is presented and tested over several different datasets. The technique is based on the maximization of the **DM** log-likelihood function using the fixed-point iteration algorithm presented in [17] and the Languasco-Migliardi (**LM**) procedure for estimating the log-likelihood function for overdispersed count data [13]. To evaluate the performance of the proposed technique, experiments using several overdispersed datasets were conducted using the **LM** procedure and the log-likelihood estimating procedure depicted in [28], denoted as **YS**. The results of the experiments were used to compare the performance of these procedures in terms of time to converge, number of iterations, and goodness of estimates. The experimental results showed the superiority of the **LM** procedure both in term of efficiency, and in term of accuracy.

One aspect of this work that can be further studied is the utilization of other ways for computing the paired differences of values of the special functions $\text{loggamma}(\log \Gamma)$ and $\text{digamma}(F)$. Here, in order to implement Algorithm 1, Python's `scipy.digamma` function was utilized coupled with our pure python implementations of the $\log \mathcal{L}$ functions of both Yu-Shaw and Languasco-Migliardi. However, an alternative approach worthy of exploring would be implementing this algorithm using the C-Python wrapped **LM**-based version of the $\log \mathcal{L}$ function. Additionally, another set of experiments can be conducted to compare Python's `scipy.digamma` and `loggamma` functions to C-Python wrapped **LM**-based versions of both functions. Further experiments can be conducted after implementing Algorithm 1 in C and comparing pure C versions of the `digamma` and `loggamma` functions to pure C **LM**-based versions of these two functions.

Another aspect that is worthy of study is the effect of using alternative numerical procedures, to estimate the **DM**

distribution parameters, on both efficiency and accuracy. While this study utilized the fixed-point iteration algorithm, it would be worthwhile to evaluate other approaches for maximizing the log-likelihood of the distribution.

Finally, the LM-approach can work anytime a ratio between values of the Euler's Gamma function (or a difference between values of the log Γ function) are present; for example, it should be possible to insert it into the Smooth Dirichlet Multinomial Models, as defined in [18] and [26].

ACKNOWLEDGMENT

The authors would like to thank Prof. Eva Riccomagno (University of Genova) for her valuable insights on the Distance metrics.

REFERENCES

- [1] L. Anderlucci and C. Viroli, "Mixtures of Dirichlet-Multinomial distributions for supervised and unsupervised classification of short text data," *Adv. Data Anal. Classification*, vol. 14, pp. 759–770, 2020, doi: [10.1007/s11634-020-00399-3](#).
- [2] A. Benton and M. Dredze, "Deep Dirichlet multinomial regression," in *Proc. 2018 Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2018, pp. 365–374, doi: [10.18653/v1/N18-1034](#).
- [3] S. Burkhardt and S. Kramer, "Decoupling sparsity and smoothness in the Dirichlet variational autoencoder topic model," *J. Mach. Learn. Res.*, vol. 20, no. 131, pp. 1–27, 2019. [Online]. Available: <https://jmlr.csail.mit.edu/papers/volume20/18-569/18-569.pdf>
- [4] X. Chen et al., "Recurrent neural network language model adaptation for multi-genre broadcast speech recognition," in *Proc. 16th Annu. Conf. Int. Speech Commun. Assoc.*, Dresden, Germany, 2015, pp. 3511–3515, doi: [10.21437/Interspeech.2015-696](#).
- [5] M. Daghyani, N. Zamzami, and N. Bouguila, "Efficient computation of log-likelihood function in clustering overdispersed count data using multinomial Beta-Liouville distribution," in *Proc. 2019 IEEE Symp. Ser. Comput. Intell.*, Xiamen, China, 2019, pp. 986–993, doi: [10.1109/SSCI44817.2019.9003076](#).
- [6] C. Elkan, "Clustering documents with an exponential-family approximation of the Dirichlet compound multinomial distribution," in *Proc. 23rd Int. Conf. Mach. Learn.*, 2006, pp. 289–296, doi: [10.1145/1143844.1143881](#).
- [7] M. Frean, A. Friedlander, M. Johnston-Hollitt, and C. Hollitt, "Source detection in astronomical images by Bayesian model comparison," in *Proc. AIP Conf.*, 2014, pp. 55–61, doi: [10.1063/1.4903710](#).
- [8] X. Gao, H. Lin, and Q. Dong, "A Dirichlet-Multinomial Bayes classifier for disease diagnosis with microbial compositions," *mSphere*, vol. 2, no. 6, 2017, Art. no. e00536-17, doi: [10.1128/mSphereDirect.00536-17](#).
- [9] J. Huete, L. Campos, J. M. Fernandez-Luna, and M. A. Rueda Morales, "Using structural content information for learning user profiles," in *Proc. 30th Special Int. Group Inf. Retrieval*, Amsterdam, 2007, pp. 38–45. [Online]. Available: <https://api.semanticscholar.org/CorpusID:16137741>
- [10] M. Hutter, "Sparse adaptive Dirichlet-Multinomial-like processes," in *Proc. 26th Annu. Conf. Learn. Theory*, 2013, pp. 432–459. [Online]. Available: <http://proceedings.mlr.press/v30/Hutter13.pdf>
- [11] M. D. Koslovsky, "A Bayesian zero-inflated Dirichlet-multinomial regression model for multivariate compositional count data," *Biometrics*, vol. 79, pp. 3239–3251, 2023, doi: [10.1111/biom.13853](#).
- [12] M. Kull, M. P. Nieto, M. Kängsepp, T. S. Filho, H. Song, and P. Flach, "Beyond temperature scaling: Obtaining well-calibrated multi-class probabilities with Dirichlet calibration," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, 2019, Art. no. 1103, doi: [10.5555/3454287.3455390](#).
- [13] A. Languasco and M. Migliardi, "On the fast computation of the Dirichlet-multinomial log-likelihood function," *Comput. Statist.*, vol. 38, pp. 1995–2013, 2023, doi: [10.1007/s00180-022-01311-7](#).
- [14] R. Lara-Cabrera, A. González, F. Ortega, and A. González-Prieto, "Dirichlet matrix factorization: A reliable classification-based recommender system," *Appl. Sci.*, vol. 12, no. 3, 2022, Art. no. 1223, doi: [10.3390/app12031223](#).
- [15] Z. Li et al., "Individualized passenger travel pattern multi-clustering based on graph regularized tensor latent Dirichlet allocation," *Data Mining Knowl. Discov.*, vol. 36, pp. 1247–1278, 2022, doi: [10.1007/s10618-022-00842-3](#).
- [16] W. Li, J. Yin, and H. Chen, "Supervised topic modeling using hierarchical Dirichlet process-based inverse regression: Experiments on e-commerce applications," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 6, pp. 1192–1205, Jun. 2018, doi: [10.1109/TKDE.2017.2786727](#).
- [17] T. P. Minka, "Estimating a Dirichlet distribution," 2000. Revised: 2003, 2009, 2012. [Online]. Available: <https://tminka.github.io/papers/dirichlet/minka-dirichlet.pdf>
- [18] R. Nallapati, T. P. Minka, and S. Robertson, "The Smoothed-Dirichlet distribution: A new building block for generative topic models," 2007. [Online]. Available: https://www.cs.cmu.edu/nmramesh/sd_tc.pdf
- [19] B. S. Oboh and N. Bouguila, "Unsupervised learning of finite mixtures using scaled Dirichlet distribution and its application to software modules categorization," in *Proc. 2017 IEEE Int. Conf. Ind. Technol.*, Toronto, ON, Canada, 2017, pp. 1085–1090, doi: [10.1109/ICIT.2017.7915513](#).
- [20] M. H. Rahman and N. Bouguila, "Efficient feature mapping in classifying proportional data," *IEEE Access*, vol. 9, pp. 3712–3724, 2021, doi: [10.1109/ACCESS.2020.3047536](#).
- [21] J. M. Roldán, O. P. Vila, and J. V. Marca, "Dirichlet uncertainty wrappers for actionable algorithm accuracy accountability and auditability," in *Proc. 2020 Conf. Fairness Accountability Transparency*, 2020, pp. 581–581, doi: [10.1145/3351095.3372825](#).
- [22] D. Shen, C. Qin, C. Wang, Z. Dong, H. Zhu, and H. Xiong, "Topic modeling revisited: A document graph-based neural network perspective," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 14681–14693. [Online]. Available: <https://openreview.net/pdf?id=yewqeLly5D8>
- [23] M. Sklar, "Fast MLE computation for the Dirichlet multinomial," 2014. Revised: 2023, *arXiv:1405.0099*. [Online]. Available: [10.48550/arXiv.1405.0099](#)
- [24] S. Syed and M. Spruit, "Selecting priors for latent Dirichlet allocation," in *Proc. 2018 IEEE 12th Int. Conf. Semantic Comput.*, Laguna Hills, CA, USA, 2018, pp. 194–202, doi: [10.1109/ICSC.2018.00035](#).
- [25] L. Wickramasinghe, A. Leblanc, and S. Muthukumarana, "Model-based estimation of baseball batting metrics," *J. Appl. Statist.*, vol. 48, no. 10, pp. 1775–1797, 2020, doi: [10.1080/02664763.2020.1775792](#).
- [26] L. Wickramasinghe, A. Leblanc, and S. Muthukumarana, "Smoothed Dirichlet distribution," *J. Statist. Theory Appl.*, vol. 22, pp. 237–261, 2023, doi: [10.1007/s44199-023-00062-8](#).
- [27] P. Yu et al., "Integrated analysis of a compendium of RNA-Seq datasets for splicing factors," *Sci. Data*, vol. 7, 2020, Art. no. 178, doi: [10.1038/s41597-020-0514-7](#).
- [28] P. Yu and C. A. Shaw, "An efficient algorithm for accurate computation of the Dirichlet-multinomial log-likelihood function," *Bioinformatics*, vol. 30, pp. 1547–1554, 2014, doi: [10.1093/bioinformatics/btu079](#).
- [29] N. Zamzami, R. Alsuroji, O. Eromonsele, and N. Bouguila, "Proportional data modeling via selection and estimation of a finite mixture of scaled Dirichlet distributions," *Comput. Intell.*, vol. 36, no. 2, pp. 459–485, 2020, doi: [10.1111/coin.12246](#).
- [30] N. Zamzami and N. Bouguila, "Probabilistic modeling for frequency vectors using a flexible shifted-scaled Dirichlet distribution prior," *ACM Trans. Knowl. Discov. Data*, vol. 14, pp. 1–35, 2020, doi: [10.1145/3406242](#).
- [31] X. Zhao, J. Zhang, and W. Lin, "Clustering multivariate count data via Dirichlet-multinomial network fusion," *Comput. Statist. Data Anal.*, vol. 179, 2023, Art. no. 107634, doi: [10.1016/j.csda.2022.107634](#).
- [32] B. Zhao, Y. Zhong, G. S. Xia, and L. Zhang, "Dirichlet-derived multiple topic scene classification model for high spatial resolution remote sensing imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 4, pp. 2108–2023, Apr. 2016, doi: [10.1109/TGRS.2015.2496185](#).



Sherenaz Al-Haj Baddar received the PhD degree in computer science from Kent State University, USA, back in 2009, then joined the University of Jordan as an assistant professor from 2009 until 2016, and as an associate professor since 2016. In 2014, she joined the Centro Interuniversitario di ricerca sull'Ingegneria delle Piattaforme Informatiche—CIPI Laboratory, University of Genoa, Italy, to pursue a post-doctoral study in anomaly detection in wireless networks. Her research interests span anomaly/intrusion detection in computer networks, IoT networks security, and parallel and distributed computing. She co-authored several peer-reviewed papers and books, and tutored more than 100 bachelor and master students with the University of Jordan.



Alessandro Languasco received the master's (summa cum laude) degree in mathematics from the University of Genova, in 1989, and the philosophiae doctor degree in mathematics from the University and Politecnico of Torino and the University of Genova, in 1994. After having enjoyed some Research Grants from the Consiglio Nazionale delle Ricerche, and a postdoctoral position with the University of Genova, in 1998 he earned a faculty position with the University of Padova, first as assistant professor and then as associate professor. In 2003, he was one of the recipients of the Distinguished Award of the Hardy-Ramanujan Society. He is a member of the Università di Padova Unit for the TRACE4EU Project (Digital Europe Programme) and of the CIPI (Centro Interuniversitario di ricerca sull'Ingegneria delle Piattaforme Informatiche). His research interests include elementary, analytic and computational number theory, cryptography, special functions, and numerical analysis. He has authored four textbooks, and about seventy research papers.



Mauro Migliardi received the laurea degree in electronic engineering and the PhD degree in computer engineering from the University of Genoa. He was a research associate, assistant professor, and adjunct professor with the University of Genoa, and research associate with Emory University. Currently, he is associate professor with the University of Padua. He is a member of the Scientific Committee of the Center for Computing Platforms Engineering, he has won the 2013 Canada-Italy Innovation Reward and he leads the joint CIPI-Gruppo Sigla s.r.l. Lab for CyberSecurity. He participated to several national and international research projects, most recently the EU funded TRACE4EU Project, Topic IP DIGITAL-2022-DEPLOY-02-EBSI-SERVICES, Digital Europe Programme, where he is PI for the Università di Padova Unit. His main research interest include distributed systems engineering in general; recently he focused on SSI, mobile systems, cybersecurity, green security, IoT security, and human memory support services. He has tutored more than 100 among bachelor, master and PhD students with the Universities of Genoa, Padua and Emory, and he has authored or co-authored more than 150 scientific papers published in national and international, peer reviewed conferences, books, and journals.